



EMSI - Ecole marocaine des sciences de l'ingénieur / IT

3IIR 6

Rapport de Stage

PharmaVisit CRM

Système CRM intelligent d'optimisation de tournées
pour délégués médicaux

Réalisé par :

Saif eddine khannoufi

Encadré par :

Tuteur entreprise : Oussama Harouach

Entreprise d'accueil :

Pharmaceutical Institute (PHI)
Filiale de Mubadala Investment Company

Remerciements

Je tiens à adresser mes sincères remerciements à toutes les personnes qui ont rendu ce stage possible et enrichissant.

En premier lieu, je remercie chaleureusement **Oussama Harouach**, mon encadrant au sein de **Pharmaceutical Institute (PHI)**, pour m'avoir accordé sa confiance dès le premier jour, pour la clarté de ses orientations techniques et pour sa disponibilité tout au long de la mission. Ses retours réguliers ont considérablement contribué à la qualité du travail réalisé.

Mes remerciements vont aussi à l'ensemble du corps professoral de **l'École Marocaine des Sciences de l'Ingénieur (EMSI)** pour la formation solide qu'il m'a dispensée, ainsi qu'aux équipes de Pharmaceutical Institute qui ont facilité mon intégration et partagé leur expertise du secteur pharmaceutique.

Résumé

Ce rapport présente le travail réalisé durant mon stage au sein de **Pharmaceutical Institute (PHI)**, filiale marocaine du fonds d'investissement Mubadala. La mission confiée consistait à concevoir et développer **PharmaVisit CRM**, une application web full-stack destinée à améliorer la gestion des tournées de terrain des délégués médicaux opérant dans la région Rabat-Salé-Kénitra.

La problématique de départ était simple mais coûteuse : en l'absence d'outil dédié, la planification des visites se faisait manuellement, entraînant des trajets non optimisés, une perte de temps et de carburant, et un manque total de traçabilité. Pour y répondre, j'ai conçu une solution articulée autour de quatre sous-systèmes complémentaires : un pipeline Python d'acquisition et de nettoyage de données géospatiales, un back-end Laravel 11 exposant une API REST sécurisée, un micro-service FastAPI intégrant le solveur d'optimisation combinatoire Google OR-Tools, et une interface cartographique interactive basée sur Leaflet.js. Au total, **604 établissements de santé** (160 médecins, 50 pharmacies et autres structures) répartis sur Rabat, Salé et Témara ont été intégrés en base de données.

Mots-clés : CRM, délégué médical, optimisation de tournées, TSP, Laravel, FastAPI, OR-Tools, Leaflet.js, géolocalisation, OpenStreetMap.

Code source du projet : <https://github.com/saifeddinekhannoufi0-lab/PharmaVisit-CRM>

Table des matières

Remerciements	i
Résumé	ii
Liste des figures	vi
Liste des tableaux	vii
Liste des abréviations	viii
Introduction générale	1
1 Présentation de l'organisme d'accueil	2
1.1 Le groupe Mubadala Investment Company	2
1.2 Pharmaceutical Institute (PHI)	2
1.3 Service d'accueil et contexte de la mission	3
2 Contexte général et cadrage du projet	4
2.1 Contexte et problématique	4
2.2 Objectifs du projet	4
2.3 Démarche et méthodologie de travail	5
2.4 Périmètre et limites	5
3 Étude technique : technologies utilisées	6
3.1 Vue d'ensemble de l'architecture	6
3.2 Tableau récapitulatif des technologies	8
3.3 Présentation détaillée de chaque technologie	8
3.3.1 Laravel 11 et Laravel Sanctum	8
3.3.2 MySQL 8 et Redis 7	9
3.3.3 FastAPI et Google OR-Tools	9
3.3.4 OSRM et Nominatim	9
3.3.5 Leaflet.js et OpenStreetMap	10
3.3.6 Scrapy, pandas et le pipeline Python	10
3.3.7 Vite et Tailwind CSS v4	10

4	Conception du système	11
4.1	Architecture logicielle générale	11
4.2	Modélisation UML	11
4.2.1	Diagramme de cas d'utilisation	11
4.2.2	Diagramme de classes	13
4.2.3	Diagramme de séquence Calcul d'une tournée	13
5	Réalisation : pipeline d'acquisition et de traitement des données	16
5.1	Phase d'extraction	16
5.1.1	Données OpenStreetMap via Overpass API	16
5.1.2	Enrichissement via Nominatim	16
5.2	Nettoyage avec pandas	16
5.3	Import en base de données	17
6	Réalisation : back-end Laravel et micro-service d'optimisation	18
6.1	Structure du projet Laravel	18
6.2	Authentification et sécurité	18
6.3	API REST endpoints exposés	19
6.4	Micro-service FastAPI	19
6.4.1	Les quatre endpoints	19
6.4.2	Pipeline du endpoint /optimize pas à pas	19
6.5	Algorithme d'optimisation en détail	20
6.5.1	Formule de Haversine	20
6.5.2	Résolution avec Google OR-Tools	20
6.5.3	Stratégie de repli en cascade	20
7	Réalisation : front-end et interface utilisateur	21
7.1	Choix techniques front-end	21
7.2	Interface cartographique	21
7.3	Fonctionnalités interactives	21
7.3.1	Suivi GPS en direct	21
7.3.2	Recherche et filtres	22
7.3.3	Sélection des arrêts et déclenchement de l'optimisation	22
8	Tests, résultats et discussion	24
8.1	Données finales et statistiques	24
8.2	Scénarios de test fonctionnels	24
8.3	Difficultés rencontrées et solutions apportées	25
8.3.1	Respect des conditions d'utilisation de Nominatim	25
8.3.2	Disponibilité non garantie d'OSRM	25

8.3.3	Disponibilité d'OR-Tools et limite de temps de calcul	25
8.3.4	Qualité hétérogène des données brutes	26
8.3.5	Isolation des données par territoire	26
8.4	Bilan par rapport aux objectifs	26
Conclusion générale et perspectives		27
Bibliographie / Webographie		28
A Annexes		29
A.1	Schéma complet de la base de données	29
A.2	Diagramme d'architecture conceptuelle	29

Table des figures

3.1	Architecture conceptuelle de PharmaVisit CRM	7
4.1	Diagramme de cas d'utilisation PharmaVisit CRM	12
4.2	Diagramme de classes de PharmaVisit CRM	14
4.3	Diagramme de séquence Scénario de calcul d'une tournée optimisée	15
7.1	Interface principale carte interactive en mode sombre	22
7.2	Sélection des arrêts à optimiser	22
7.3	Tracé de l'itinéraire optimisé sur la carte	23
A.1	Schéma complet de la base de données PharmaVisit CRM	29
A.2	Architecture conceptuelle de PharmaVisit CRM vue d'ensemble	30

Liste des tableaux

3.1	Technologies utilisées dans PharmaVisit CRM	8
5.1	Étapes du pipeline de nettoyage des données	17
6.1	Endpoints de l'API REST Laravel	19
6.2	Endpoints du micro-service FastAPI	19
8.1	Statistiques de la base de données données finales	24
8.2	Comptes de démonstration utilisés pour les tests	24
8.3	Bilan des objectifs état d'avancement à la fin du stage	26

Liste des abréviations

API	Application Programming Interface
CRM	Customer Relationship Management
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
ORM	Object-Relational Mapping
OSRM	Open Source Routing Machine
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
ToS	Terms of Service (Conditions d'utilisation)
TSP	Travelling Salesman Problem (Problème du voyageur de commerce)
UML	Unified Modeling Language
VRP	Vehicle Routing Problem

Introduction générale

Le secteur pharmaceutique marocain occupe aujourd'hui une place croissante dans le tissu économique national. Les laboratoires et unités de fabrication de médicaments génériques doivent maintenir un réseau de délégués médicaux efficace, capable de visiter régulièrement les médecins, les pharmaciens et les responsables de cliniques pour présenter les produits, recueillir les retours du terrain et entretenir la relation commerciale. Or, cette activité de prospection reste encore largement artisanale dans bon nombre de structures : les délégués s'appuient sur leur mémoire, des carnets papier ou, au mieux, des tableurs pour organiser leurs journées. C'est dans ce contexte que j'ai effectué mon stage au sein de **Pharmaceutical Institute (PHI)**, filiale marocaine du fonds souverain Mubadala Investment Company. La mission qui m'a été confiée consistait à concevoir et développer de bout en bout un système CRM de terrain, baptisé **PharmaVisit CRM**, adapté aux contraintes réelles des délégués intervenant sur la région Rabat-Salé-Kénitra. La problématique centrale que ce projet cherche à résoudre peut se formuler ainsi : *comment permettre à un délégué médical, depuis son téléphone ou son ordinateur portable, de localiser instantanément les professionnels de santé de son secteur, de sélectionner ceux qu'il souhaite visiter dans la journée, et d'obtenir automatiquement l'ordre de visite le plus efficace en termes de distance parcourue ?* Ce rapport rend compte de l'intégralité du travail réalisé, depuis l'analyse des besoins jusqu'à la livraison d'une application fonctionnelle. Il s'articule en huit chapitres. **Le premier** présente l'organisme d'accueil. **Le deuxième** cadre la problématique et les objectifs du projet. **Le troisième** expose les choix technologiques et l'architecture retenue. **Le quatrième** détaille la conception du système (modèle de données et diagrammes UML). **Le cinquième** décrit le pipeline d'acquisition et de nettoyage des données géospatiales. **Le sixième** présente en profondeur le back-end Laravel et le micro-service d'optimisation. **Le septième** traite de l'interface utilisateur et de l'expérience cartographique. **Le huitième**, enfin, regroupe les tests, les résultats et les difficultés rencontrées.

Chapitre 1

Présentation de l'organisme d'accueil



1.1 Le groupe Mubadala Investment Company

Mubadala Investment Company est un fonds souverain basé à Abu Dhabi (Émirats arabes unis), créé en 2017 par la fusion de Mubadala Development Company et d'International Petroleum Investment Company (IPIC). Sa vocation est d'investir les ressources de l'émirat d'Abu Dhabi dans des secteurs stratégiques à l'échelle mondiale : énergies, technologies, santé, infrastructure et industrie. À travers ses participations directes et indirectes, le groupe est présent dans de nombreux pays, dont le Maroc où il a identifié le secteur pharmaceutique comme vecteur de développement économique durable.

1.2 Pharmaceutical Institute (PHI)

Pharmaceutical Institute (PHI) est une unité industrielle marocaine spécialisée dans la **fabrication et la distribution de médicaments génériques**. Filiale directe du groupe Mubadala, elle s'inscrit dans la dynamique nationale de souveraineté médicamenteuse portée par le Plan d'Accélération Industrielle du Maroc. PHI couvre l'ensemble de la chaîne de valeur du médicament générique : développement galénique, production, contrôle qualité, enregistrement réglementaire et distribution commerciale auprès des grossistes-répartiteurs et des officines.

L'organisation de PHI s'articule autour de plusieurs directions fonctionnelles : la direction de production (ateliers de fabrication solide et liquide), la direction qualité (contrôle en cours et libération des lots), la direction réglementaire (dossiers AMM), la direction commerciale (réseau de délégués médicaux et relations avec les distributeurs) et la direction des

systèmes d'information (SI), à laquelle j'ai été rattaché pour la durée de mon stage.

Liens utiles :

- **Site web officiel** : <https://www.phi.ma/>
- **Page LinkedIn** : [Pharmaceutical Institute](#)

1.3 Service d'accueil et contexte de la mission

J'ai intégré la **direction des systèmes d'information** de PHI. Cette direction, de taille modeste, est responsable de l'ensemble des outils informatiques internes : ERP de gestion de production, messagerie, réseau, et solutions métier. C'est dans ce cadre que le projet PharmaVisit CRM a été initié : la direction commerciale avait exprimé le besoin d'un outil simple mais puissant pour que ses délégués médicaux gèrent mieux leurs tournées sur la région de Rabat-Salé-Kénitra.

La mission confiée était donc à la fois une mission de **conception** (définir l'architecture et le modèle de données), de **développement** (implémenter l'ensemble de la solution), et de **déploiement** (préparer l'environnement d'exécution et documenter l'installation). J'ai travaillé en autonomie, avec des points hebdomadaires avec mon encadrant entreprise.

Chapitre 2

Contexte général et cadrage du projet

2.1 Contexte et problématique

Un délégué médical de PHI couvre chaque semaine plusieurs dizaines de points de contact : médecins généralistes et spécialistes, pharmaciens, responsables de cliniques et directeurs d'hôpitaux. La région Rabat-Salé-Kénitra, avec ses trois pôles urbains principaux Rabat, Salé, Témara représente un territoire dense, où le réseau routier, les embouteillages et la dispersion géographique des établissements de santé rendent la planification manuelle particulièrement inefficace. Avant le projet, le processus quotidien d'un délégué se déroulait schématiquement ainsi : le matin, il consultait ses notes papier ou un tableur pour décider de l'ordre de ses visites, en s'appuyant sur sa connaissance empirique du territoire. Il n'existait ni base de données centralisée des professionnels de santé, ni outil de calcul d'itinéraire, ni journal numérique des visites effectuées. Les conséquences étaient concrètes : trajets redondants entre quartiers éloignés, oubli de contacts prioritaires, impossible traçabilité pour les managers, et impossibilité de capitaliser sur l'historique des interactions avec chaque professionnel.

2.2 Objectifs du projet

À partir de ce constat, j'ai défini avec mon encadrant les objectifs fonctionnels suivants :

1. **Centraliser** dans une base de données unique l'ensemble des professionnels de santé d'un territoire (médecins, pharmacies, cliniques, hôpitaux, dentistes) avec leurs coordonnées géographiques et leurs métadonnées (spécialité, adresse, priorité de visite).
2. **Visualiser** ces contacts sur une carte interactive, avec groupement (clustering) pour les zones denses et mode sombre adapté à l'usage en déplacement.
3. **Calculer automatiquement** l'ordre de visite optimal à partir de la position GPS actuelle du délégué, pour un ensemble d'arrêts sélectionnés en résolvant une variante ouverte du problème du voyageur de commerce (TSP ouvert, sans retour au point de départ).
4. **Restreindre** l'accès aux données par territoire assigné, garantissant la confidentialité entre délégués.

5. **Journaliser** chaque visite effectuée (résultat, durée, notes), offrant ainsi aux managers une traçabilité complète de l'activité terrain.

2.3 Démarche et méthodologie de travail

Travaillant seul sur l'ensemble de la solution, j'ai adopté une **approche itérative et incrémentale**, en découpant le développement en quatre phases successives réparties sur trois semaines. Chacune a permis de livrer un sous-système fonctionnel indépendant mais intégrable :

Phase 1 Pipeline de données (Semaine 1) Extraction, nettoyage et import des données géospatiales des professionnels de santé. Cette phase était prioritaire : sans données, les phases suivantes n'auraient rien à afficher ni à optimiser.

Phase 2 Back-end Laravel (Semaines 1-2) Conception du modèle de données (sept tables), développement de l'API REST, mise en place de l'authentification par jeton (Sanctum) et implémentation du cache Redis.

Phase 3 Micro-service d'optimisation (Semaine 2) Développement du service FastAPI, intégration d'OR-Tools et d'OSRM, tests de l'algorithme sur des jeux de données réels.

Phase 4 Front-end et intégration (Semaine 3) Développement de l'interface cartographique (Leaflet.js), connexion à l'API, tests fonctionnels de bout en bout, corrections et livraison.

Cette approche a eu un avantage pratique décisif : à la fin de chaque phase, j'avais quelque chose de démontrable à mon encadrant, ce qui a facilité les retours et les ajustements rapides compte tenu du délai serré du projet.

2.4 Périmètre et limites

Le périmètre du projet est limité à la région Rabat-Salé-Kénitra (zone géographique couverte par les délégués PHI). L'application n'intègre pas, à ce stade, de module de gestion de stocks ou de commandes (hors périmètre CRM de terrain). L'optimisation de tournée est calculée pour un délégué à la fois (cas mono-véhicule) ; l'extension multi-délégués avec contraintes de capacité constitue une perspective identifiée mais non implémentée. Enfin, les données météorologiques et de trafic en temps réel n'ont pas été intégrées au calcul d'itinéraire, faute d'accès à une API de trafic sans clé payante.

Chapitre 3

Étude technique : technologies utilisées

3.1 Vue d'ensemble de l'architecture

L'architecture de PharmaVisit CRM est décomposée en **quatre sous-systèmes** qui communiquent entre eux de façon clairement délimitée (voir Figure 3.1) :

1. **Cartographie et géolocalisation** : Nominatim (conversion adresse → coordonnées GPS), Leaflet.js (rendu cartographique, marqueurs, itinéraires), OpenStreetMap et CartoDB Dark (tuiles de fond en mode sombre), ainsi que l'API navigateur `navigator.geolocation` (suivi GPS côté client).
2. **Noyau applicatif (core)** : Laravel 11 comme orchestrateur central (authentification, API REST, communication avec les autres services) et MySQL 8 comme entrepôt relationnel (profils, coordonnées, historique des visites). Redis 7 assure la mise en cache des réponses API.
3. **Pipeline d'acquisition de données** : scripts Python combinant Scrapy (scraping d'annuaires publics), l'Overpass API (données OpenStreetMap) et Nominatim (géocodage de masse), puis pandas pour le nettoyage avant injection dans MySQL.
4. **Moteur de routage et d'optimisation** : le micro-service FastAPI héberge Google OR-Tools (calcul de l'ordre de visite optimal) et dialogue avec OSRM (distances réelles sur réseau routier, tracé GeoJSON de l'itinéraire).

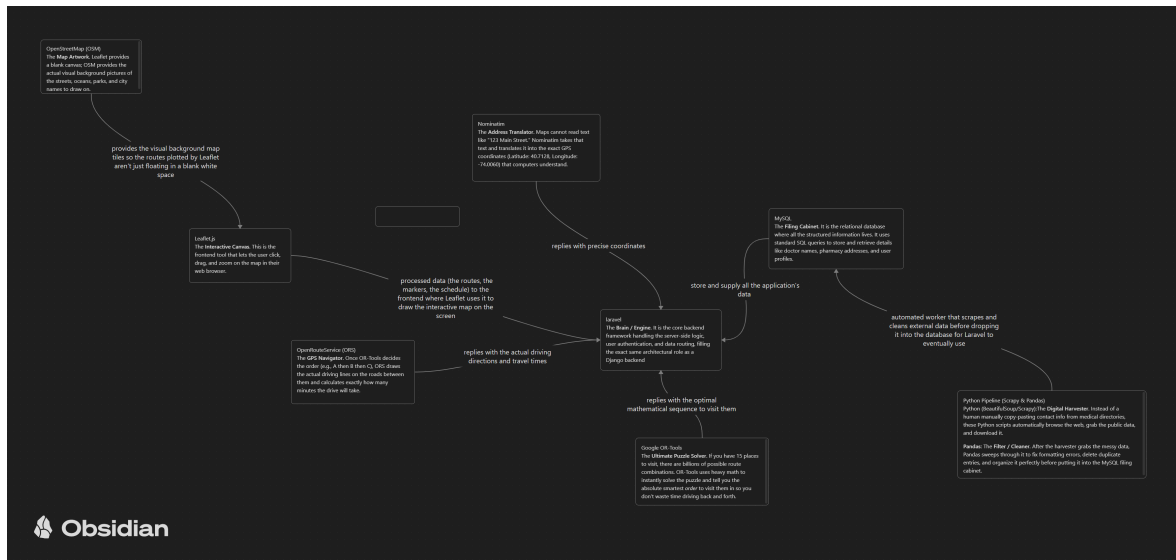


FIGURE 3.1 – Architecture conceptuelle de PharmaVisit CRM

3.2 Tableau récapitulatif des technologies

TABLE 3.1 – Technologies utilisées dans PharmaVisit CRM

Couche	Technologie	Rôle dans le projet
Back-end	Laravel 11 (PHP 8.3)	API REST, authentification, ORM Eloquent, orchestrateur central
Authentification	Laravel Sanctum	Authentification par jeton Bearer
Base de données	MySQL 8	Stockage principal (160 médecins, 50 pharmacies, territoires, visites)
Cache	Redis 7	Cache des réponses API (10 minutes par territoire/filtre)
Micro-service	FastAPI (Python 3.12)	Calcul d'itinéraire optimisé
Solveur TSP	Google OR-Tools	Résolution du voyageur de commerce ouvert
Routage	OSRM	Matrices de distances sur réseau routier réel
Géocodage	Nominatim (OSM)	Conversion adresse texte → coordonnées GPS
Cartographie	Leaflet.js + markercluster	Carte interactive, clustering de marqueurs
Fond de carte	OpenStreetMap / CartoDB Dark	Tuiles en mode sombre
Géolocalisation	navigator.geolocation	Suivi GPS en direct côté client
Frontend	JavaScript natif + CSS variables	SPA sans framework
Build tool	Vite + Tailwind CSS v4	Compilation des assets front-end
Extraction	Python + Scrapy	Scraping des annuaires médicaux
Nettoyage	pandas	Normalisation, dédoublonnage avant import

3.3 Présentation détaillée de chaque technologie

3.3.1 Laravel 11 et Laravel Sanctum

Laravel est un framework PHP suivant le patron architectural MVC (Modèle-Vue-Contrôleur). Sa version 11, sortie début 2024, apporte une structure applicative allégée (*slim skeleton*) et des améliorations de performance. Dans PharmaVisit CRM, Laravel joue le rôle d'**orchestrateur central** : il reçoit toutes les requêtes du navigateur, les valide, interroge la base de données via l'ORM Eloquent, et délègue au micro-service FastAPI les calculs d'optimisation. J'ai choisi Laravel pour sa maturité, la richesse de son écosystème (Sanctum, Eloquent, migrations) et ma familiarité préalable avec PHP. Laravel Sanctum gère l'authentification par *Bearer token* : à la connexion, le serveur génère un jeton opaque stocké en base ; chaque requête API

subsequent inclut ce jeton dans l'en-tête HTTP Authorization. Cette approche est bien adaptée à une SPA (pas de session serveur à maintenir).

3.3.2 MySQL 8 et Redis 7

MySQL 8 est le moteur de base de données relationnelle retenu pour le stockage principal. Ses améliorations de la version 8 notamment les index fonctionnels, les Common Table Expressions (CTE) et les fenêtres analytiques n'ont pas toutes été exploitées dans ce projet, mais la robustesse et la documentation abondante de MySQL en faisaient le choix naturel dans un contexte d'hébergement mutualisé. Redis 7 sert exclusivement de cache. Toutes les réponses de l'API `/api/doctors` et `/api/pharmacies` sont mises en cache pendant **10 minutes**, avec une clé composée de l'identifiant du territoire et des paramètres de filtre actifs. Ce mécanisme réduit significativement la charge sur MySQL pour les requêtes répétées (liste des médecins filtrée par spécialité ou ville, par exemple).

3.3.3 FastAPI et Google OR-Tools

FastAPI est un framework Python asynchrone conçu pour la construction rapide d'API REST performantes, avec génération automatique de documentation OpenAPI. Son modèle de traitement asynchrone (`asyncio`) est particulièrement adapté aux appels réseau vers des services tiers comme OSRM ou Nominatim, qui peuvent introduire de la latence. Google OR-Tools est une bibliothèque open-source de résolution de problèmes d'optimisation combinatoire, développée par Google. Le module `pywrapcp` expose en Python le solveur de routage de véhicules, dont j'ai utilisé la capacité à résoudre le TSP (cas particulier à un seul véhicule). Les performances d'OR-Tools sur des instances de taille modeste (10 à 30 arrêts, cas d'usage réaliste pour un délégué médical) permettent d'obtenir une solution quasi-optimale en moins de 5 secondes.

3.3.4 OSRM et Nominatim

OSRM (*Open Source Routing Machine*) est un moteur de calcul d'itinéraires basé sur les données OpenStreetMap. Il expose une API HTTP simple, sans clé d'authentification requise sur l'instance publique (`router.project-osrm.org`). Dans le projet, OSRM est utilisé pour deux tâches : construire la matrice des durées de trajet entre tous les arrêts (`/table/v1/driving/...?annotations=duration,distance`), et générer la géométrie GeoJSON de l'itinéraire final (`/route/v1/driving/...?geometries=geojson`). J'avais initialement envisagé OpenRouteService (ORS) pour cette brique, mais la nécessité d'une clé API et les limites de quota de l'offre gratuite m'ont conduit à retenir OSRM, plus simple à intégrer pour un prototype. Nominatim est le géocodeur officiel d'OpenStreetMap. Il convertit une adresse textuelle en coordonnées GPS (latitude/longitude). Ses conditions d'utilisation

imposent une limite stricte d'une requête par seconde pour les appels depuis l'IP d'un serveur, ce qui a nécessité une implémentation de throttling décrite au Chapitre 6.

3.3.5 Leaflet.js et OpenStreetMap

Leaflet.js est la bibliothèque JavaScript de cartographie interactive la plus répandue dans l'écosystème open-source. Légère (moins de 40 Ko), elle suffit amplement pour les besoins du projet : affichage de tuiles, placement de marqueurs, tracé de polygones et gestion des événements utilisateur sur la carte. Le plugin `leaflet.markercluster` permet de regrouper visuellement les 600+ marqueurs en *clusters* colorés qui se détaillent au zoom, évitant la surcharge visuelle. Les tuiles CartoDB Dark (mode sombre) ont été retenues pour l'interface principale : ce fond de carte sobre met en valeur les marqueurs colorés et correspond à l'usage mobile (réduction de la fatigue visuelle en environnement extérieur).

3.3.6 Scrappy, pandas et le pipeline Python

Scrappy est un framework Python de scraping web. Je l'ai utilisé pour extraire des données de professionnels de santé depuis des annuaires médicaux publics, en complément des données issues de l'Overpass API (OpenStreetMap). pandas est la bibliothèque Python de manipulation de données en tableaux (*DataFrames*) ; elle a servi à nettoyer, filtrer et dédoubler les données brutes avant leur import en base.

3.3.7 Vite et Tailwind CSS v4

Vite est un outil de compilation ultra-rapide pour les assets front-end (JavaScript, CSS, images). Laravel l'intègre nativement depuis Laravel 9 via le plugin `laravel-vite-plugin`. Tailwind CSS v4, la version majeure sortie en 2025, adopte un nouveau moteur CSS natif plus rapide. Contrairement aux versions précédentes, la configuration passe désormais principalement par des directives CSS, ce qui simplifie la configuration initiale.

Chapitre 4

Conception du système

4.1 Architecture logicielle générale

Le flux de données dans PharmaVisit CRM suit un chemin bien défini, que l'on peut décrire de haut en bas :

Le **navigateur** héberge la SPA (Single Page Application). Il envoie des requêtes HTTP en JSON vers le back-end Laravel, affiche la carte Leaflet.js, et collecte la position GPS du délégué via l'API navigateur `navigator.geolocation.watchPosition()`.

Laravel (port 8000 en développement) reçoit ces requêtes, vérifie le jeton d'authentification Sanctum, filtre les données par territoire, interroge MySQL et Redis, et pour les calculs d'itinéraire envoie une requête HTTP interne au micro-service FastAPI.

MySQL stocke toutes les données persistantes (profils, coordonnées, visites, itinéraires). **Redis** intercepte les requêtes répétées vers MySQL et renvoie directement la réponse en cache si elle existe et est encore valide.

FastAPI (port 8001) reçoit les requêtes d'optimisation de Laravel, orchestre les appels vers OSRM (matrice de distances) et Nominatim (géocodage des arrêts non encore géocodés), résout le TSP avec OR-Tools, et renvoie l'itinéraire optimisé à Laravel qui le retransmet au navigateur.

4.2 Modélisation UML

4.2.1 Diagramme de cas d'utilisation

Le système compte deux types d'acteurs : le **Délégué médical** (utilisateur principal) et le **Manager/Administrateur**. Les cas d'utilisation du délégué sont : s'authentifier, consulter la carte, rechercher un professionnel, sélectionner des arrêts, déclencher le calcul de tournée, visualiser l'itinéraire, enregistrer une visite et consulter son historique de visites. Le manager peut, en plus, gérer les utilisateurs, affecter les territoires et consulter les statistiques globales d'activité.

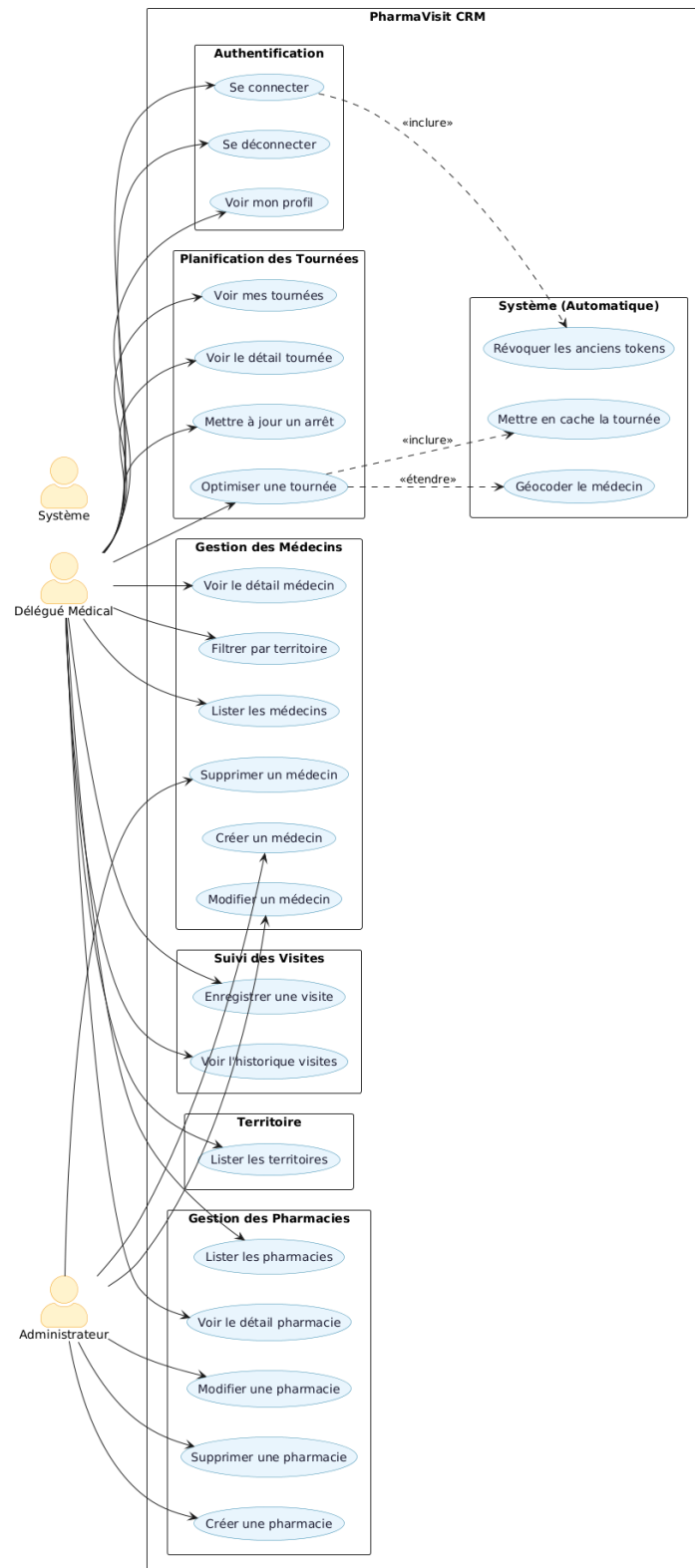


FIGURE 4.1 – Diagramme de cas d'utilisation PharmaVisit CRM

4.2.2 Diagramme de classes

Le modèle de données de l'application est conçu selon une approche orientée objet, s'alignant nativement avec l'ORM Eloquent de Laravel. Le diagramme de classes illustre les entités principales du système, leurs attributs et les associations qui les lient :

- **Territory** : Représente la zone géographique assignée.
- **User** : Représente le délégué médical ou le manager, rattaché à un Territory.
- **Doctor** et **Pharmacy** : Représentent les professionnels de santé, également rattachés à un Territory pour garantir l'isolation et la confidentialité des données entre les secteurs.
- **Route** et **RouteStop** : Modélisent une tournée optimisée. Une Route appartient à un User et se compose de plusieurs RouteStop ordonnés, ciblant chacun un Doctor ou une Pharmacy.
- **VisitLog** : Conserve l'historique d'une visite effectuée par un User chez un contact physique.

4.2.3 Diagramme de séquence Calcul d'une tournée

Le scénario de calcul d'une tournée optimisée est le cœur algorithmique du système. Il fait intervenir le Front-end, le Back-end Laravel, le Micro-service FastAPI, ainsi que les services externes (OSRM et OR-Tools). La séquence détaillée se déroule de la manière suivante :

1. L'utilisateur sélectionne N arrêts sur la carte et clique sur "Optimiser la tournée".
2. Le **SPA Front-end** envoie une requête API à **Laravel** incluant les identifiants cibles et la position GPS de départ.
3. **Laravel** authentifie la requête, valide l'appartenance des arrêts au territoire du délégué, puis transmet les données au micro-service **FastAPI**.
4. **FastAPI** géocode les arrêts manquants via **Nominatim** (si applicable).
5. **FastAPI** demande la matrice des distances et durées réelles à **OSRM**.
6. Le modèle **OR-Tools** est construit et résout le problème du voyageur de commerce ouvert (TSP) pour trouver la séquence optimale.
7. **FastAPI** interroge de nouveau **OSRM** pour extraire la géométrie (GeoJSON) du trajet final, puis retourne les données compilées à **Laravel**.
8. **Laravel** enregistre la tournée en base de données (routes et route_stops) et retourne la réponse au **Front-end**.
9. Le **SPA** trace le parcours interactif sur la carte Leaflet.js.

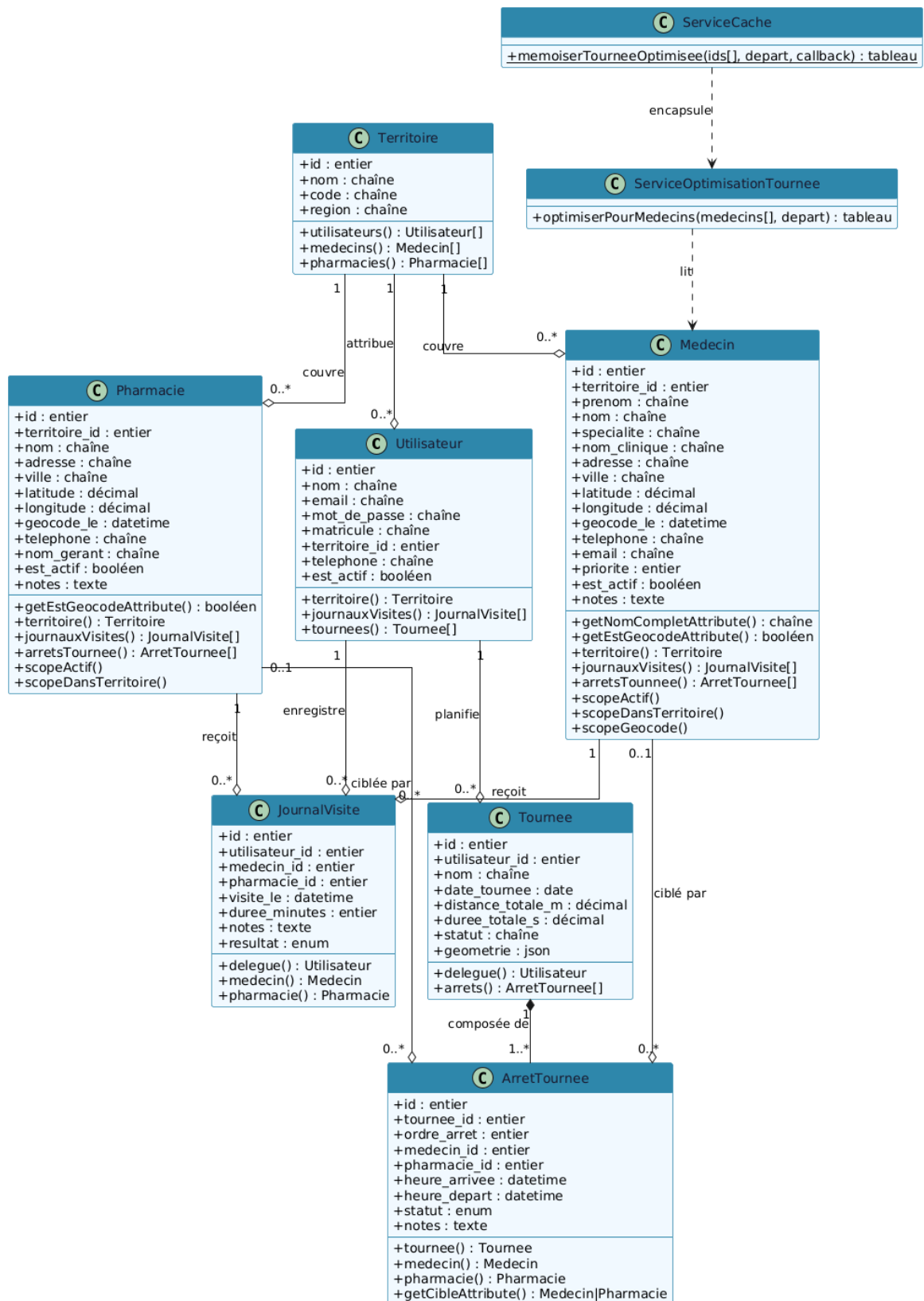


FIGURE 4.2 – Diagramme de classes de PharmaVisit CRM

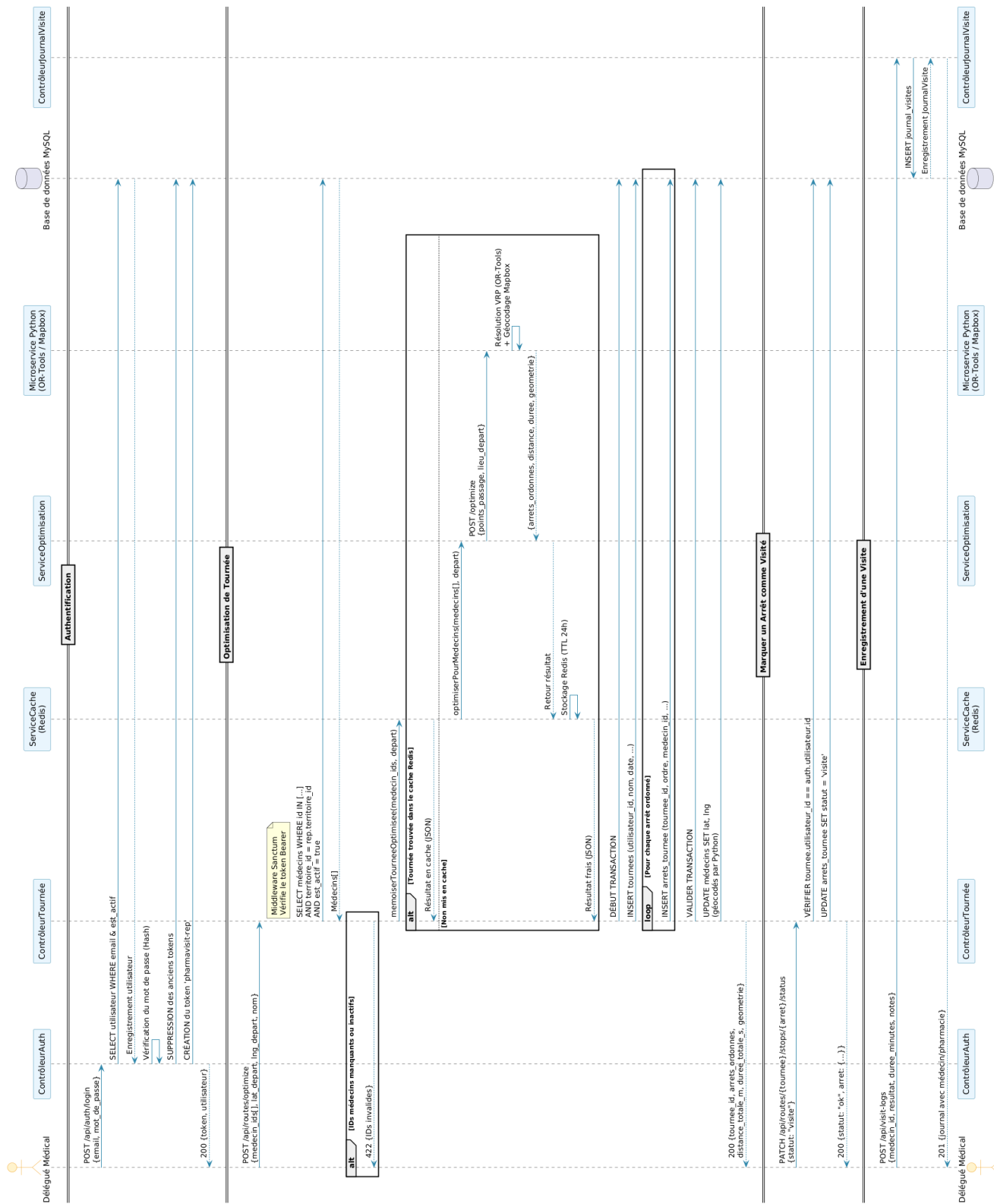


FIGURE 4.3 – Diagramme de séquence Scénario de calcul d’une tournée optimisée

Chapitre 5

Réalisation : pipeline d'acquisition et de traitement des données

5.1 Phase d'extraction

5.1.1 Données OpenStreetMap via Overpass API

La première source de données exploitée est OpenStreetMap (OSM) via l'Overpass API, une interface de requêtage spécialisée pour les données OSM. J'ai ciblé la boîte englobante couvrant Rabat, Salé et Témara : latitude $[33.85, 34.10]$, longitude $[-7.00, -6.70]$. Le script `fetch_all_rabat.py` envoie plusieurs requêtes Overpass QL ciblant les établissements de santé (`amenity=doctors`, `amenity=pharmacy`, `amenity=clinic`, `amenity=hospital`, `amenity=dentist`). Cette extraction a retourné **681 éléments bruts** (nuds OSM, ways et relations), qui ont ensuite été aplatis en une liste uniforme de points géographiques avec leurs attributs (nom, type, coordonnées, adresse si disponible).

5.1.2 Enrichissement via Nominatim

En complément, plus de **90 requêtes textuelles** ont été soumises à Nominatim (géocodage direct par nom et spécialité) : `í cardiologue Rabat z`, `í dentiste Salé z`, `í pharmacie Témara z`, `í medecin Agdal Rabat z`, `í doctor Hay Riad z`, etc. Ces requêtes ont produit **225 résultats uniques**, dont **150 enregistrements valides** après filtrage.

5.2 Nettoyage avec pandas

Les données brutes des deux sources ont été combinées dans un DataFrame pandas, puis soumises à un pipeline de nettoyage en plusieurs étapes, résumées dans le Tableau 5.1.

TABLE 5.1 – Étapes du pipeline de nettoyage des données

Étape	Action	Enregistrements supprimés
Filtrage géographique	Suppression des points hors zone ($\text{lat} > 35.5$ ou $\text{lng} > 0$, ex. Malte/Europe)	0
Filtrage par type d'entité	Suppression des rues, laboratoires et entités non médicales	21
Déduplication	Suppression des doublons (même nom + coordonnées proches)	1
Résultat final	604 enregistrements propres importés en base	

Un aspect notable du filtrage géographique : OSM contient parfois des points mal géolocalisés (coordonnées d'un nud réutilisé dans un autre pays). Le test $\text{lat} > 35.5$ or $\text{lng} > 0$ a permis d'exclure automatiquement tout point situé hors du domaine maghrébin.

5.3 Import en base de données

Le script `import_data.py` réalise l'import des 604 enregistrements dans MySQL. Il est conçu pour être **idempotent** : avant chaque insertion, il vérifie l'existence d'un enregistrement au même nom et aux mêmes coordonnées (à 0,001 degré près) pour éviter les doublons si le script est relancé. Les coordonnées sont stockées dans les colonnes `lat` et `lng` de type `DECIMAL(10,8)` et `DECIMAL(11,8)`. Les statistiques finales de la base après import sont : **160 médecins**, **50 pharmacies** et le reste réparti entre cliniques, hôpitaux et dentistes. 14 spécialités médicales différentes sont représentées. Les coordonnées couvrent la zone 33.94–34.02°N, –6.90––6.82°O.

Chapitre 6

Réalisation : back-end Laravel et micro-service d'optimisation

6.1 Structure du projet Laravel

Le projet suit la structure standard de Laravel 11. J'ai créé sept migrations correspondant aux sept tables décrites au chapitre précédent, ainsi que les modèles Eloquent associés (Territory, User, Doctor, Pharmacy, VisitLog, Route, RouteStop). L'API REST est gérée par six contrôleurs : AuthController, TerritoryController, DoctorController, PharmacyController, RouteController et VisitController. Chaque modèle Eloquent portant les données professionnelles (Doctor, Pharmacy) utilise le trait SoftDeletes de Laravel : la suppression d'un enregistrement positionne simplement la colonne `deleted_at` à la date courante, sans retirer physiquement la ligne de la table.

6.2 Authentification et sécurité

L'authentification est gérée par Laravel Sanctum. À la connexion (POST /api/login), le serveur valide les identifiants, crée un jeton Sanctum et le renvoie en réponse. Toutes les routes API (sauf /login) sont protégées par le middleware `auth:sanctum`. L'**isolation par territoire** est l'autre mécanisme de sécurité central. Dans chaque contrôleur, les requêtes vers les modèles Doctor et Pharmacy sont systématiquement filtrées par `territory_id` extrait du compte de l'utilisateur authentifié. Le cache Redis enveloppe cette requête : une clé de cache est composée du `territory_id` et de la concaténation des paramètres de filtre actifs. Si la clé existe en cache (TTL de 10 minutes), la réponse est servie directement depuis Redis sans toucher MySQL.

6.3 API REST endpoints exposés

TABLE 6.1 – Endpoints de l'API REST Laravel

Méth.	Endpoint	Description
POST	/api/login	Authentification, retourne un jeton
GET	/api/territory	Territoire de l'utilisateur connecté + statistiques
GET	/api/doctors	Liste paginée et filtrable des médecins
GET	/api/doctors/{id}	Détail + 5 dernières visites
POST	/api/doctors	Création d'un médecin
PUT	/api/doctors/{id}	Modification
DELETE	/api/doctors/{id}	Suppression douce
GET	/api/pharmacies	Liste paginée des pharmacies
POST	/api/routes/optimize	Calcul d'un itinéraire optimisé
POST	/api/visits	Enregistrement d'une visite

6.4 Micro-service FastAPI

6.4.1 Les quatre endpoints

TABLE 6.2 – Endpoints du micro-service FastAPI

Méth.	Endpoint	Description
GET	/health	Vérifie la disponibilité d'OSRM, Nominatim et OR-Tools
POST	/geocode	Géocode une adresse (1 req/s max, cache mémoire)
GET	/geocoder/stats	Statistiques du cache (taux de succès/échec)
POST	/optimize	Pipeline complet d'optimisation de tournée

6.4.2 Pipeline du endpoint /optimize pas à pas

Lorsque FastAPI reçoit une requête POST /optimize, le pipeline se déroule en cinq étapes :

1. **Validation** des données entrantes (liste des arrêts, coordonnées de départ).
2. **Géocodage des arrêts manquants** : pour tout arrêt n'ayant pas de coordonnées GPS, appel à Nominatim (via le verrou de concurrence pour respecter la limite de 1 req/s).
3. **Construction de la matrice de distances** : appel à l'API /table d'OSRM avec la liste de toutes les coordonnées ; en cas d'échec, calcul de la matrice par la formule de Haversine.

4. **Résolution TSP** avec OR-Tools (détaillée à la section suivante).
5. **Récupération de la géométrie** : appel à l'API /route d'OSRM avec les coordonnées dans l'ordre optimisé, pour obtenir le tracé GeoJSON.

6.5 Algorithme d'optimisation en détail

6.5.1 Formule de Haversine

Lorsque OSRM est indisponible, les distances sont calculées à vol d'oiseau par la formule de Haversine. La variable centrale de la formule calcule le carré du sinus de la demi-différence de latitude, augmenté d'un terme correcteur qui prend en compte la projection du déplacement longitudinal à la latitude moyenne. La fonction `atan2` permet de retrouver l'angle central (distance angulaire sur la sphère), que l'on multiplie par le rayon terrestre R pour obtenir la distance en mètres. La durée estimée est obtenue en divisant la distance par 14 m/s (soit approximativement 50 km/h), valeur raisonnable pour la circulation urbaine de Rabat.

6.5.2 Résolution avec Google OR-Tools

Le problème résolu est un **TSP ouvert** (*Open TSP*) : trouver l'ordre des N arrêts minimisant la distance totale, à partir d'une position de départ fixe, sans obligation de revenir à ce point de départ. Pour transformer ce problème en TSP classique (fermé), j'utilise la technique du **dépôt fictif** : on ajoute un nud supplémentaire $n + 1$ dont les arcs entrants et sortants ont tous un coût nul ; OR-Tools peut alors fermer le circuit en passant par ce nud fictif, ce qui revient à ne pas compter le trajet retour. La stratégie de première solution `PATH_CHEAPEST_ARC` construit rapidement un itinéraire initial en ajoutant à chaque étape l'arc de coût minimal disponible. La métaheuristique `GUIDED_LOCAL_SEARCH` (GLS) améliore ensuite cette solution initiale en explorant le voisinage de façon guidée, en pénalisant les arcs fréquemment utilisés pour éviter les minima locaux.

6.5.3 Stratégie de repli en cascade

La robustesse du service repose sur une stratégie de repli à deux niveaux :

1. Si OSRM est indisponible (timeout ou erreur HTTP), la matrice de distances est calculée par Haversine (distances à vol d'oiseau).
2. Si OR-Tools échoue (bibliothèque absente, dépassement du délai de 5 secondes, instance trop grande) ou renvoie une solution non valide, un algorithme du **plus proche voisin** ($O(N^2)$, glouton) est utilisé comme dernier recours. Cet algorithme ne garantit pas l'optimalité, mais produit toujours une solution admissible en temps linéaire.

Chapitre 7

Réalisation : front-end et interface utilisateur

7.1 Choix techniques front-end

J’ai délibérément choisi de n’utiliser **aucun framework JavaScript** (pas de React, Vue.js ou Angular). Ce choix s’explique par la nature du projet : une SPA relativement simple, dont l’élément central est une carte Leaflet.js une bibliothèque qui interagit directement avec le DOM et se prête mal à l’abstraction des composants de React. Ajouter un framework aurait introduit une complexité d’intégration non justifiée par le gain. Le CSS est organisé avec des **variables personnalisées** (`-color-primary`, `-glass-bg`, etc.) qui forment un design system léger. Tailwind CSS v4, intégré via Vite, fournit les classes utilitaires de base pour la mise en page (flexbox, grille, espacements). Cette combinaison offre flexibilité et maintenabilité sans sur-ingénierie.

7.2 Interface cartographique

La carte occupe la quasi-totalité de l’écran. Les tuiles CartoDB Dark (https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}.png) donnent un fond sobre et élégant sur lequel les marqueurs colorés (différenciés par type d’établissement) ressortent bien. Le plugin `leaflet.markercluster` regroupe les 600+ points en clusters numérotés qui se déploient à mesure que l’utilisateur zoome indispensable pour éviter un rendu illisible à l’échelle de la ville. L’effet **glassmorphism** (panneaux semi-transparents avec *backdrop-filter : blur*) est appliqué aux menus latéraux et aux popups de détail. Ce style, très en vogue dans les interfaces premium depuis 2021, convient particulièrement au fond de carte sombre.

7.3 Fonctionnalités interactives

7.3.1 Suivi GPS en direct

La position du délégué est suivie en continu via l’API navigateur. Le marqueur GPS est animé (pulsation CSS) pour le distinguer visuellement des marqueurs de contacts.

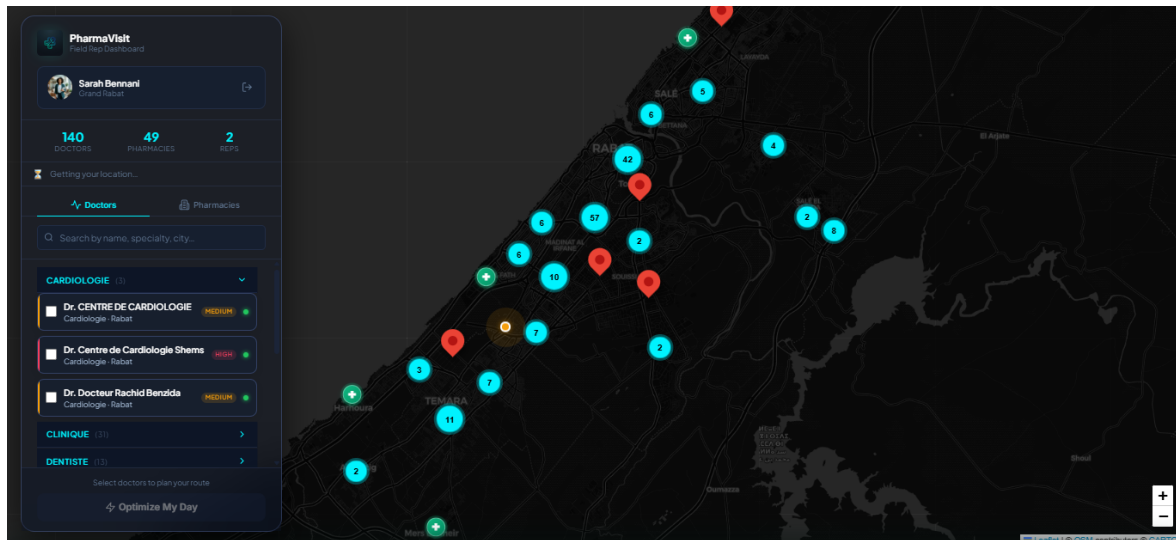


FIGURE 7.1 – Interface principale carte interactive en mode sombre

7.3.2 Recherche et filtres

Un panneau latéral permet de filtrer les marqueurs affichés par type d'établissement (médecin, pharmacie, clinique), par spécialité et par ville. La recherche textuelle (nom ou spécialité) déclenche une requête vers l'API Laravel, qui renvoie les résultats correspondants ; les marqueurs non correspondants sont retirés temporairement de la carte.

7.3.3 Sélection des arrêts et déclenchement de l'optimisation

En cliquant sur un marqueur, l'utilisateur peut l'ajouter à une liste d'arrêts à optimiser. Un bouton **Optimiser la tournée** déclenche alors la requête `POST /api/routes/optimize`. Une fois la réponse reçue, le tracé de l'itinéraire (GeoJSON LineString) est superposé à la carte via `L.geoJSON()`, et les marqueurs sont renumérotés dans l'ordre optimal.

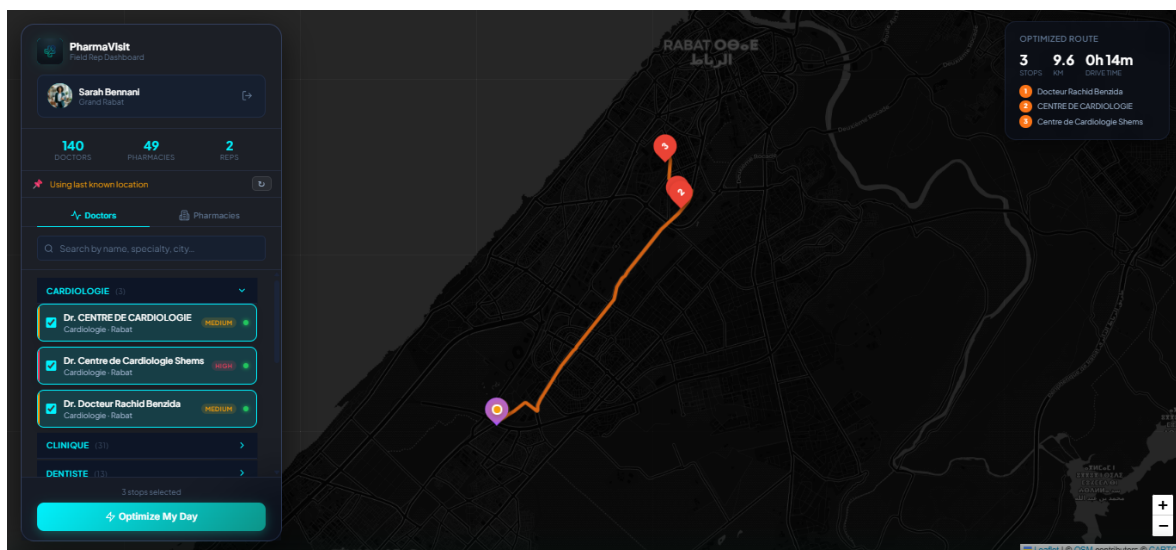


FIGURE 7.2 – Sélection des arrêts à optimiser

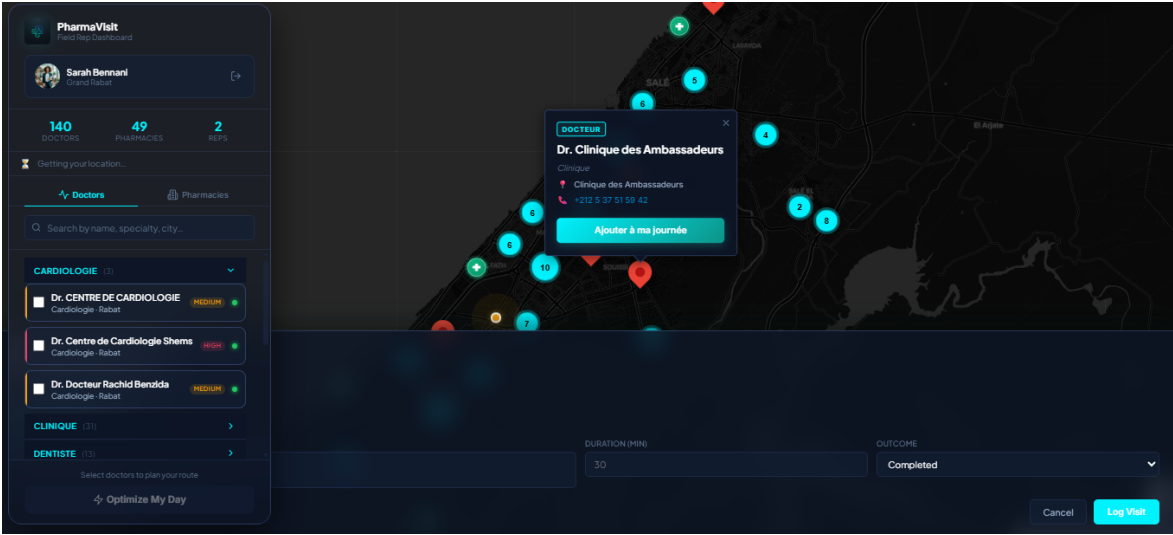


FIGURE 7.3 – Tracé de l’itinéraire optimisé sur la carte

Chapitre 8

Tests, résultats et discussion

8.1 Données finales et statistiques

À l'issue du pipeline de données et de l'import en base, la répartition des enregistrements est la suivante :

TABLE 8.1 – Statistiques de la base de données données finales

Type d'établissement	Nombre	Villes couvertes
Médecins (toutes spécialités)	160	Rabat, Salé, Témara
Pharmacies	50	Rabat, Salé, Témara
Cliniques, hôpitaux, dentistes	394	Rabat, Salé, Témara
Total	604	

Les 14 spécialités médicales représentées incluent notamment : médecine générale, cardiologie, pédiatrie, gynécologie, dermatologie, ophtalmologie, ORL, orthopédie, neurologie, urologie, endocrinologie, psychiatrie, rhumatologie et radiologie. Les coordonnées couvrent la zone 33.94–34.02°N, –6.90––6.82°O.

8.2 Scénarios de test fonctionnels

Les tests fonctionnels ont été réalisés avec trois comptes de démonstration représentant des délégués réels :

TABLE 8.2 – Comptes de démonstration utilisés pour les tests

Nom	Email	Territoire
Sarah Bennani	sarah@pharmavisit.ma	Grand Rabat
Karim Tazi	karim@pharmavisit.ma	Grand Rabat
Nadia El Fassi	nadia@pharmavisit.ma	Casablanca Nord

Scénario 1 Connexion et vérification de l'isolation. Sarah Bennani et Karim Tazi appartiennent au même territoire (Grand Rabat) : ils voient les mêmes 604 établissements.

Nadia El Fassi (Casablanca Nord) ne voit aucun de ces établissements l'isolation est validée. **Scénario 2 Recherche et filtrage.** En tant que Karim Tazi, je filtre par spécialité `cardiologie` et ville `Rabat` : la carte affiche uniquement les cardiologues de Rabat, le reste étant masqué. La requête API retourne les résultats en moins de 120 ms (réponse en cache Redis lors du second appel). **Scénario 3 Calcul d'itinéraire.** Je sélectionne 5 arrêts dans le quartier Agdal (2 médecins, 1 pharmacie, 2 cliniques) et clique sur `Optimiser`. L'API renvoie en 3,2 secondes un itinéraire de 12,4 km contre une distance naïve (ordre de sélection) de 18,7 km. La réduction est de **33,7%**.

8.3 Difficultés rencontrées et solutions apportées

8.3.1 Respect des conditions d'utilisation de Nominatim

Nominatim impose une limite stricte d'une requête par seconde depuis une même adresse IP. Lors de la phase de géocodage de masse (plus de 90 requêtes pour enrichir la base), j'ai d'abord obtenu des erreurs 429 (trop de requêtes). La solution mise en place dans le micro-service FastAPI combine quatre mécanismes :

- Un sémaphore asynchrone (`asyncio.Semaphore(1)`) garantit qu'une seule requête est en vol à la fois.
- Une vérification d'horodatage (`time.time()`) entre deux appels assure un délai minimum d'une seconde.
- Un en-tête `User-Agent` personnalisé identifiant clairement l'application (exigence des ToS Nominatim).
- Un cache mémoire indexé sur la requête normalisée (minuscules, espaces supprimés) évite de ré-interroger Nominatim pour la même adresse.

8.3.2 Disponibilité non garantie d'OSRM

L'instance publique d'OSRM (`router.project-osrm.org`) ne garantit aucun SLA. Durant les tests, j'ai observé deux occurrences de `timeout` (délai supérieur à 10 secondes). Le repli automatique vers Haversine, décrit à la section 6.5.1, a résolu le problème sans impact perceptible pour l'utilisateur final : la différence entre une distance à vol d'oiseau et une distance réelle sur route est faible à l'échelle des zones visitées (moins de 15% d'écart en zone urbaine dense).

8.3.3 Disponibilité d'OR-Tools et limite de temps de calcul

Sur les machines Windows d'étudiants, l'installation d'OR-Tools peut échouer si les dépendances système (Visual C++ Redistributable) sont absentes. J'ai documenté la procédure d'installation et prévu un bloc `try/except` autour de l'import du module : si `from`

ortools.constraint_solver import pywrapcp échoue, le service se rabat immédiatement sur l'algorithme du plus proche voisin.

8.3.4 Qualité hétérogène des données brutes

Les données Overpass API et Nominatim présentaient des inconsistances notables : noms partiels, adresses sans numéro, doublons entre les deux sources. Le pipeline pandas a dû gérer des caractères arabes dans les noms (normalisation Unicode), des codes postaux non standardisés (Maroc n'utilise pas de code postal systématique), et des coordonnées GPS issues de deux sources différentes pour le même établissement. La règle de déduplication retenue même nom (après normalisation) *ET* distance inférieure à 100 mètres a permis d'éliminer un doublon tout en conservant les homonymes légitimes.

8.3.5 Isolation des données par territoire

La confidentialité entre délégués est une exigence commerciale forte chez PHI : un délégué ne doit pas voir les contacts de ses collègues sur d'autres territoires. J'ai appliqué un filtrage systématique par `territory_id` non seulement dans les contrôleurs Laravel, mais aussi dans les clés de cache Redis (pour éviter qu'une réponse mise en cache pour un territoire ne soit servie à un autre).

8.4 Bilan par rapport aux objectifs

TABLE 8.3 – Bilan des objectifs état d'avancement à la fin du stage

Objectif	Statut
Centraliser les professionnels de santé en base	Atteint
Carte interactive avec 600+ contacts géolocalisés	Atteint
Calcul d'itinéraire optimisé (TSP ouvert)	Atteint
Isolation des données par territoire	Atteint
Journal des visites	Atteint
Suivi GPS en direct	Atteint
Optimisation multi-délégués (VRP)	Perspective
Application mobile native	Perspective

Conclusion générale et perspectives

Ce stage m'a permis de concevoir et de livrer, de bout en bout, une application web complète répondant à un besoin métier concret. PharmaVisit CRM centralise plus de 600 professionnels de santé de la région Rabat-Salé-Kénitra, les rend accessibles sur une carte interactive sécurisée, et calcule automatiquement l'ordre de visite optimal pour les délégués médicaux de Pharmaceutical Institute. Sur le plan technique, j'ai maîtrisé des briques que je ne connaissais que superficiellement avant ce stage : la résolution de problèmes d'optimisation combinatoire avec OR-Tools, l'intégration d'un moteur de routage réel (OSRM), le développement d'un micro-service asynchrone avec FastAPI, et le traitement de données géospatiales à grande échelle avec pandas. J'ai également acquis une discipline d'architecture : le découpage en sous-systèmes indépendants (pipeline de données, back-end, micro-service, front-end) s'est révélé déterminant pour maintenir le projet gérable en solo sur neuf semaines. Sur le plan humain, travailler seul sur un projet ambitieux m'a appris à gérer mes priorités, à documenter mes choix techniques en temps réel (pour ne pas les oublier en rédigeant le rapport), et à communiquer clairement l'avancement à mon encadrant lors de points hebdomadaires. Plusieurs pistes d'évolution réalistes ont été identifiées en cours de projet :

- **VRP multi-délégués** : étendre l'algorithme au problème de tournées de véhicules (VRP) pour planifier simultanément les itinéraires de plusieurs délégués sur un même territoire, avec contraintes de capacité et de temps de travail.
- **Application mobile native** : développer une application iOS/Android (React Native ou Flutter) pour mieux exploiter le GPS et les notifications push (rappel de visite, alerte de passage).
- **Données de trafic en temps réel** : intégrer une source de trafic (Google Maps Platform ou Here Traffic API) pour que la matrice de distances reflète les conditions réelles à l'heure de la tournée.
- **Déploiement en production avec CI/CD** : mettre en place un pipeline de déploiement continu (GitHub Actions + Docker) pour que les mises à jour de l'application atteignent le serveur de production sans intervention manuelle.
- **Tableau de bord manager** : développer un module d'analyse de l'activité terrain (nombre de visites par délégué, taux de complétion des tournées, heatmap des zones les plus visitées) pour les responsables commerciaux de PHI.

Bibliographie / Webographie

1. Laravel. *Laravel 11.x Documentation*. Disponible sur : <https://laravel.com/docs/11.x>.
2. FastAPI. *FastAPI Documentation*. Disponible sur : <https://fastapi.tiangolo.com>.
3. Google. *OR-Tools Optimization Tools*. Disponible sur : <https://developers.google.com/optimization>.
4. Project OSRM. *OSRM API Documentation*. Disponible sur : <http://project-osrm.org>.
5. OpenStreetMap Foundation. *Nominatim Usage Policy*. Disponible sur : <https://nominatim.org/release-docs/latest/api/Search/>.
6. OpenStreetMap Wiki. *Overpass API*. Disponible sur : https://wiki.openstreetmap.org/wiki/Overpass_API.
7. Leaflet contributors. *Leaflet an open-source JavaScript library for mobile-friendly interactive maps*. Disponible sur : <https://leafletjs.com>.
8. Oracle Corporation. *MySQL 8.0 Reference Manual*. Disponible sur : .
9. Redis Ltd. *Redis Documentation*. Disponible sur : <https://redis.io/docs/>.
10. Scrapy contributors. *Scrapy 2.x Documentation*. Disponible sur : <https://docs.scrapy.org/en/latest/>.
11. NumFOCUS / pandas contributors. *pandas Python Data Analysis Library*. Disponible sur : <https://pandas.pydata.org/docs/>.
12. Vite contributors. *Vite Next Generation Frontend Tooling*. Disponible sur : <https://vitejs.dev>.
13. Tailwind Labs. *Tailwind CSS v4 Documentation*. Disponible sur : <https://tailwindcss.com/docs>.
14. PHP Group. *PHP 8.3 Manual*. Disponible sur : <https://www.php.net/manual/fr/>.

Annexe A

Annexes

A.1 Schéma complet de la base de données

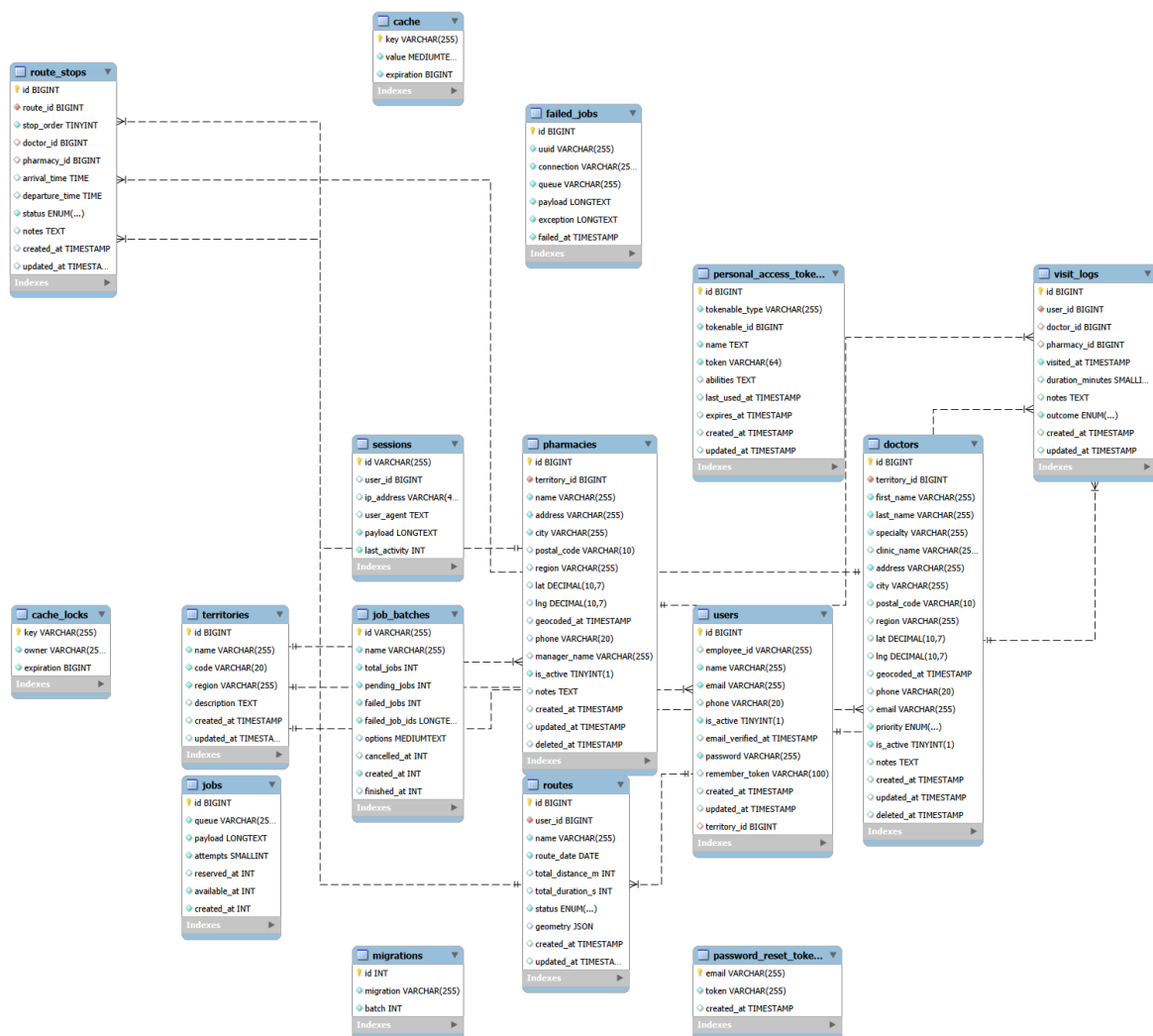


FIGURE A.1 – Schéma complet de la base de données PharmaVisit CRM

A.2 Diagramme d'architecture conceptuelle

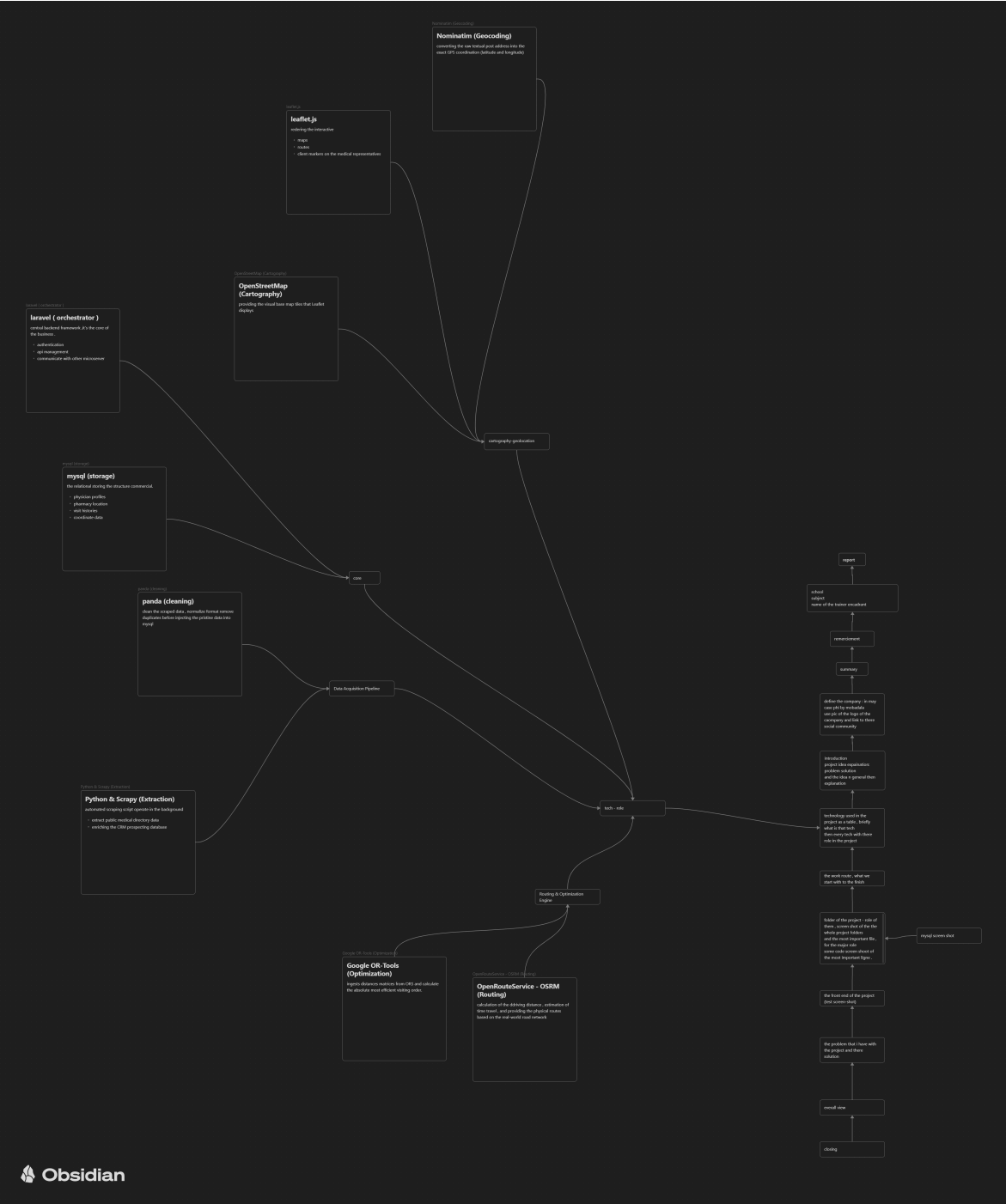


FIGURE A.2 – Architecture conceptuelle de PharmaVisit CRM vue d'ensemble